Tutorial 4:Pointers, Arrays, Strings & Streams

Yuxin Peng

123090446@link.cuhk.edu.cn

Pointers and arrays

Reading C++ declarators

A reliable reading order

1. Start at the variable name.

2. Read postfix operators on the right: [] and ().

3. Then read prefix operators on the left: *, &, and &&.

Parentheses change which operator binds first.

```
int* p1;
   p1 is a pointer to int

int** p2;
   p2 is a pointer to a pointer to int

int* a[5];
   a is an array of 5 pointers to int

int (*p)[5];
   p is a pointer to an array of 5 int

int (*p3[5])[10];
   p3 is an array of 5 pointers to arrays of 10 int
```

Arrays and pointer arithmetic

For a built-in array or pointer, a[i] is defined as *(a + i). Neither form performs bounds checking.

```
int a[5]{};

a + 1      // advances by sizeof(int)
&a + 1     // advances by sizeof(a)
           // here: 5 * sizeof(int)

sizeof(a)  // size of the complete array
```

a and &a usually hold the same starting address, but their types differ:

- a converts to int* in most expressions.
- &a has type int (*)[5].

Array-to-pointer conversion does not occur with sizeof, decltype, or unary &.

Arrays as function parameters

```
int main() {
    int a[5]{};
    std::cout << sizeof(a);
    func(a);
}

// sizeof(a) == 5 * sizeof(int)
```

```
void func(int a[]) {
    std::cout << sizeof(a);
}

// int a[] is adjusted to int* a
// sizeof(a) == sizeof(int*)
// Pointer size depends on the platform
```

Pass the element count explicitly, as in the lecture notes:
void func(int* a, std::size_t n);

Alternatives: template<std::size_t N> void func(int (&a)[N]); or std::span<int> in C++20.

Array and pointer recap

Expression	Meaning for int arr[5] and int* p = arr
arr	Converts to int* in most expressions
&arr	Pointer to the complete array: int (*)[5]
arr + 1	Address of the next int element
&arr + 1	Address immediately after the complete array
arr[i]	Equivalent to *(arr + i)
&arr[i]	Equivalent to arr + i

Strings & Streams

C-style strings and std::string

C-style string

A null-terminated sequence of char, commonly stored in a char array.

A char* is only a pointer. It represents a C string only when it points to a valid null-terminated sequence.

Two string literals cannot be concatenated with +.

std::string

Owns and manages its character storage.

Supports assignment, concatenation, comparison, and member functions.

std::string out = std::string{"hi "} + "bob";

c_str() returns const char*. The pointer can become invalid after a non-const modification.

Choose std::string for ordinary C++ text handling. Use C strings when an API requires them or when you must work with a fixed character buffer.

Core std::string operations

Expression	Meaning
s[i]	Unchecked access to character i
s.at(i)	Checked access; throws std::out_of_range when i is invalid
s1 + s2	Creates a new concatenated string
s1 += s2	Appends s2 to s1
s1 = s2	Copies s2 into s1
s1 == s2, s1 < s2	Lexicographical comparison

Useful std::string members

s.size()
Number of characters

s.substr(pos, count)
New string containing the requested range

s.find(needle, pos)
First match at or after pos, or std::string::npos

s.insert(pos, text)
Inserts text before pos

s.append(text)
Appends text

s.erase(pos, count)
Removes the requested range

s.replace(pos, count, text)
Replaces the requested range

s.c_str()
Returns a pointer to a null-terminated const char sequence

Selected <cstring> functions

std::strlen(s)
Length before the null terminator

std::strcmp(a, b)
Negative, zero, or positive result

std::strstr(text, target)
Pointer to the first match, or nullptr

std::strcpy(dest, src)
Copies src, including the null terminator

std::strcat(dest, src)
Appends src to dest

std::memset(dest, value, count)
Writes count bytes; value is converted to unsigned char

std::memcpy(dest, src, count)
Copies count bytes between non-overlapping regions

Use std::memmove when the regions overlap.

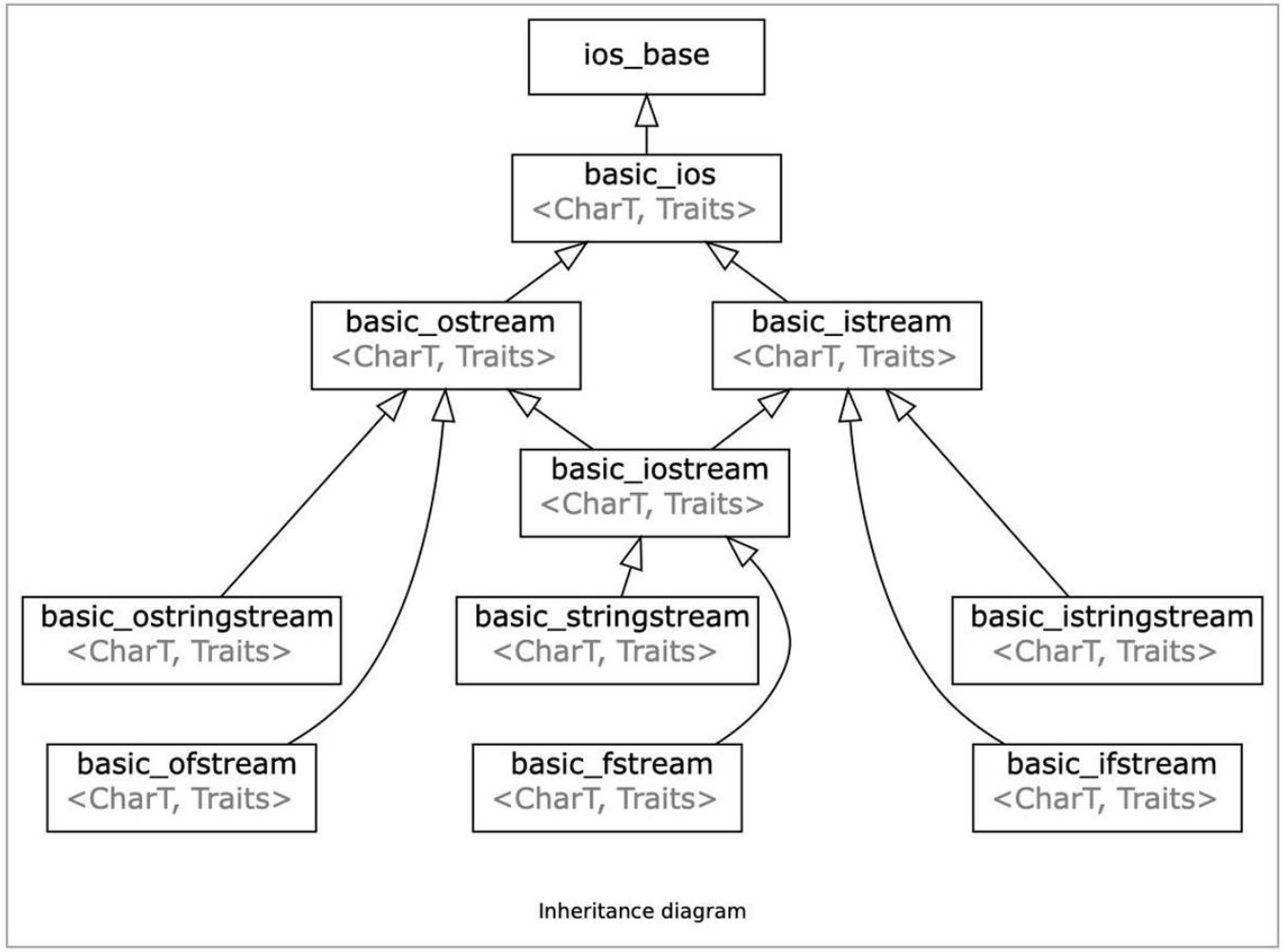
C++ stream hierarchy

A reference to a base stream class can bind to an object of a derived stream class.

• std::ostream& accepts std::cout, std::ofstream, or std::ostringstream.

• std::istream& accepts std::cin, std::ifstream, or std::istringstream.

• std::iostream supports both input and output; std::fstream and std::stringstream derive from it.



Using <cctype> safely

```
std::isalnum(ch)    letter or digit
std::isalpha(ch)    letter
std::isdigit(ch)    decimal digit
std::islower(ch)    lowercase letter
std::isupper(ch)    uppercase letter
std::tolower(ch)    lowercase conversion
std::toupper(ch)    uppercase conversion
```

The classification functions return an int that behaves as true or false. They can depend on the active C locale.

Do not replace them with ASCII range tests when you want standard character classification.

When char may be signed, convert it to unsigned char before calling these functions.

String streams and file streams

String streams <sstream>

```
std::istringstream iss{"42 3.5"};
int n;
double x;
iss >> n >> x;

std::ostringstream oss;
oss << n << " " << x;
std::string text = oss.str();
```

File streams <fstream>

```
std::ifstream in{"input.txt"};
std::ofstream out{"output.txt"};
std::fstream io{"data.txt",
    std::ios::in | std::ios::out};

if (!in) { /* open failed */ }
```

Correct stream-reading loops

Problem

```
std::string word;
while (!in.eof()) {
    in >> word;
    process(word);
}
```

Correct

```
std::string word;
while (in >> word) {
    process(word);
}

std::string line;
while (std::getline(in, line)) {
    process(line);
}
```

Palindrome check: two approaches

Compare matching positions O(n) time, O(1) extra space

```
bool isPalindrome(const std::string& s) {
    for (std::size_t i = 0; i < s.size() / 2; ++i) {
        if (s[i] != s[s.size() - 1 - i]) return false;
    }
    return true;
}
```

Reverse a copy O(n) extra space

```
bool isPalindromeCopy(const std::string& s) {
    std::string reversed = s;
    std::reverse(reversed.begin(), reversed.end());
    return reversed == s;
}
```

Exercise: acronym generator

```
std::string acronym(const std::string& phrase);
```

Requirements

- Read words with std::istringstream.
- Append the first character of each non-empty word.
- Preserve the input case, as in the lecture examples.
- Handle repeated spaces and an empty input string.

Optional: produce an uppercase acronym using std::toupper safely.

Example

Input:
GNU Compiler Collection

Output:
GCC

Questions